```
// ====================================================================
// Stripe - OLAP Physical Data Model
// Version : v22
// Format : DBML (Database Markup Language) - https://dbml.dbdiagram.io
// Cas d'usage métier : Stripe Business Case /  OLAP
// Objectif : schéma en étoile, dénormalisé, optimisé pour les requêtes d'agrégation, l'analyse de séries temporelles et le reporting.
// Devise de référence : EUR.
// v09 : précision des montants - montants locaux en unités mineures (bigint), montants EUR en decimal(18,4), taux de référence en decimal(24,12).
// v10 : géolocalisation - DimGeography alimentée par l'IP de la transaction (résolue en pays à l'ETL), membre « unknown », dimension statique.
// v11 : fuseaux horaires - référence unique UTC : horodatages en timestamptz, date_key et dates de validité = dates UTC.
// v12 : - règle de conversion et type des montants répétés dans DimReferenceExchangeRate ;
//       - frais : commission reprise de l'OLTP, jamais recalculée ; stripe_revenue_eur = colonne dérivée de fee_amount_eur ;
//         notes de DimPricingPlan alignées (règles MCD, montant facturé) ;
//       - FactAuditEvent.geography_key : source documentée (DataAccessLog.source_ip), non nulle ;
//       - coquille ìp_geolocation corrigée.
// v13 : suppression des champs sans source dans l'OLTP v36 et sans usage (D8.2, R0) :
//       DimMerchant.industry, DimProduct.category, DimActor.department, DimCustomer.lifecycle_stage,
//       DimDate.is_holiday, DimTransactionStatus.status_group ;
//       constantes (EUR) : DimReferenceExchangeRate.to_currency_key / to_currency_code, DimCurrency.is_reference_currency.
// v14 : dimensions indépendantes - aucune clé étrangère entre dimensions :
//       DimCustomer / DimProduct / DimPricingPlan : merchant_key (FK vers DimMerchant) remplacée par merchant_id (clé métier, sans lien) ;
//       DimReferenceExchangeRate : from_currency_key (FK vers DimCurrency) supprimée, from_currency_code conservé.
// v15 : corrections de cohérence (aucune décision de conception) :
//       - note du projet : commission « jamais recalculée » ;
//       - DimPaymentMethod.type (+ direct_debit), DimTransactionStatus.status_code (+ suspended) ;
//       - DimAuditAction.requires_justification ajoutée (source OLTP : AuditAction.requires_justification) ;
//       - FactFraudEvent.customer_key not null (Transaction.customer_id est not null dans l'OLTP) ;
//       - DimProduct et DimPaymentMethod : SCD1, comme en annexe A (2.A.4).
// v16 : - aucune allusion à la fonction de calcul de la commission de l'OLTP (commission reprise, jamais recalculée) ;
//       - suppression de DimCustomer.first_transaction_date et DimPricingPlan.fixed_fee_by_currency ;
//       - FactSubscriptionSnapshot : started_date_key et canceled_date_key ajoutées (DimDate, rôles multiples) ;
//         AggSubscriptionDailyMerchant : sources de new_subscriptions et churned_count précisées ;
//       - DimPricingPlan : règle MCD #22 (traçabilité du plan facturé) ajoutée aux références.
// v17 : règles de transformation (ETL) : section D3-A 2.A.9 (ETL-01 à ETL-62) ; renvois ajoutés dans les notes des colonnes calculées :
//       DimCustomer.segment, DimAuditAction.action_group, FactTransaction (is_refunded, is_chargeback, anomaly_score),
//       FactFraudEvent (is_confirmed_fraud, exposed_amount_eur), FactSubscriptionSnapshot (mrr_eur, stripe_mrr_eur, is_active, days_until_renewal),
//       AggSubscriptionDailyMerchant (compteurs, churn_rate), FactAuditEvent (was_authorized, had_justification).
// v18 : corrections de libellés (aucune décision de conception) :
//       - Subscribes.* -> Subscription.* (table OLTP) dans les notes de started_date_key / canceled_date_key ;
//       - libellés SCD : « SCD2 » conservé pour DimMerchant, DimCustomer, DimPricingPlan, DimActor ; clés des dimensions statiques,
//         de DimReferenceExchangeRate (versionnée par exercice) et des faits renommées en « Clé de substitution » ; DimCurrency : statique ;
//       - coquilles et textes périmés (« Trois grains » -> six, FactComplianceRequest, Kimbla) ;
//       - FactFraudEvent : mise à jour ultérieure de is_confirmed_fraud signalée ;
//       - points non tranchés exposés dans D3-A 2.A.9.8.
// v19 : RLS sur la clé métier (revue critique D3, §1.3) : la politique repose sur merchant_id (une merchant_key par version SCD2) ;
//       notes du projet, de DimMerchant.merchant_id et de l'indépendance des dimensions alignées. Aucune colonne modifiée.
// v20 : cohérence D3 / D3-A / D3-B (COHERENCE-D3-v18, points B) :
//       - FactSubscriptionSnapshot : currency_key et exchange_rate_key ajoutées (devise de l'abonnement, taux retenu pour mrr_eur et stripe_mrr_eur) ;
//       - index des clés métier : FactSubscriptionSnapshot (subscription_id, date_key) unique ; FactDataSubjectRequest.request_id et
//         FactSecurityIncident.incident_id uniques (ETL-03) ;
//       - AggSecurityIncidentMonthly : clé unique (merchant_key nullable) au lieu d'une clé primaire ; AggComplianceDailyMerchant : lignes sans marchand non agrégées ;
//       - notes ETL-02 complétées (status_key, refund_amount_minor, refund_amount_eur, merchant_net_revenue_eur) ;
//       - coquilles.
// v21 : alignement sur l'OLTP v36 et retrait d'une redondance :
//       - DimPaymentMethod.card_brand renommée network_code (OLTP v35 : CardBrand -> CardNetwork, PaymentMethod.network_code) ;
//       - DimMerchant.country_code et DimCustomer.country_code non nulles (Merchant.country_code, Customer.country_code non nulles en OLTP v36) ;
//       - FactSubscriptionSnapshot.currency_key retirée (v20) : exchange_rate_key désigne déjà la devise (from_currency_code).
// v22 : renvois mis à jour après la création de l'annexe D3-C (OLAP Conventions and Transformation Rules) :
```

```
//       - D3-A 2.A.7 (conversion de devise), 2.A.8 (fuseaux horaires) et 2.A.9 (règles de transformation) déplacées en D3-C
2.C.1, 2.C.2 et 2.C.3 ;
//         les mentions « D3-A 2.A.9 » des entrées v17 et v18 ci-dessus désignent donc D3-C 2.C.3 ;
//       - notes du projet et des colonnes : « D3-A 2.A.9… » devient « D3-C 2.C.3… ». Aucune colonne modifiée.
// Nomenclature : noms de tables en PascalCase, colonnes en snake_case.
// Compatible : dbdiagram.io, dbml-cli (dbml2sql).
// ====================================================================

Project StripeAnalyticsSystem {
  database_type: "PostgreSQL"
  Note: '''
    Base des données analytiques Stripe (schéma en étoile).

    Principes de conception :
    - Dénormalisation assumée : minimiser les jointures, exploiter la redondance.
    - Clés de substitution entières : compactes, rapides, requises pour le suivi historique SCD2.
    - Grain aussi fin que possible dans les faits.
    - Stockage colonne recommandé (Redshift / BigQuery / Snowflake / ClickHouse) + partitionnement par date_key.
    - SECURITE ET CONFORMITÉ : FactAuditEvent, FactDataSubjectRequest et FactSecurityIncident alimentent le reporting
réglementaire automatisé (RGPD, PCI-DSS, CCPA).

    DOUBLE DESTINATION DES ANALYSES
    Le cahier de charges précise que Stripe fournit des analyses « à ses clients et à ses parties prenantes internes ».
    L'entrepôt sert donc deux publics différents :
      - LE MARCHAND
        Besoin : son chiffre d'affaires, ses clients, ses produits.
        Mesure de référence : merchant_net_revenue_eur.
        Le système analytique est ici une composante de l'offre Stripe aux marchands - en plus d'être déjà un outil de
pilotage interne.
      - STRIPE
        Besoin : les commissions perçues, tous marchands confondus.
        Mesure de référence : stripe_revenue_eur.
    Les deux mesures cohabitent sur la même ligne de faits ; seul le choix de la colonne agrégée distingue les deux lectures.

    SÉCURITÉ AU NIVEAU DES LIGNES (row-level security)
    Conséquence directe de la destination externe : un marchand accédant au reporting ne doit voir QUE ses propres lignes.
    Le filtrage doit être imposé par le moteur (politique RLS) et ne pas être laissé à la charge des requêtes applicatives.
    La politique repose sur l'identifiant stable merchant_id, pas sur merchant_key : DimMerchant est en SCD2, donc un marchand
possède une merchant_key par version
    et une politique « merchant_key = X » lui masquerait son historique.
    - Faits et agrégats : merchant_key doit appartenir à l'ensemble des clés du marchand de la session
      (merchant_key IN (SELECT merchant_key FROM DimMerchant WHERE merchant_id = <marchand de la session>)).
    - DimMerchant, DimCustomer, DimProduct, DimPricingPlan : merchant_id égal à celui de la session. Les autres dimensions
sont des référentiels partagés.
    - Lignes sans marchand (merchant_key nul : FactAuditEvent, FactSecurityIncident) : non visibles par cette politique ;
lecture par les rôles internes à définir (D3-C 2.C.3.8, O-08).
    Tous les accès sont journalisés dans FactAuditEvent.

    ACTUALISATION DIFFÉRENCIÉE
    Le tableau de bord marchand nécessite une latence en quasi-temps réel.
    Le reporting financier interne tolère un traitement par lots nocturne.

    CONVERSION MONÉTAIRE - devise de référence : EURO.
    - Les montants sont convertis avec un TAUX DE RÉFÉRENCE FIGÉ.
        - Objectif : comparer les montants convertis entre eux.


    PRÉCISION DES MONTANTS :
    - Montants locaux : bigint
        - Entiers en unités mineures - comme OLTP.
        - Echelle = DimCurrency.decimals :
            - montant en unité majeure = amount_minor / 10^decimals.
    - Montants en EUR : decimal(18,4)
        - Décimaux exacts calculés une seule fois en ETL (jamais de flottant) :
            - amount_eur = amount_minor / 10^decimals x reference_rate
                - reference_rate = valeur en euros de 1 unité MAJEURE de la devise source.
        - Chaque composante indépendante (montant, commission, remboursement) est convertie une seule fois ;
            - La commission Stripe (fee_amount_minor) est reprise telle quelle de Transaction (fee_amount) ; elle n'est
JAMAIS recalculée côté OLAP.
            - Les mesures dérivées sont calculées sur les valeurs déjà converties :
                - merchant_net_revenue_eur = amount_eur - fee_amount_eur - refund_amount_eur.
        - Arrondi demi-supérieur à la 4e décimale ;
            - L'arrondi à la 2e décimale est réservé à l'affichage.

    GÉOLOCALISATION :
    - Source : Transaction.ip_geolocation (adresse IP, IPv4/IPv6) et Transaction.device_type. Le pays est RÉSOLU À L'ETL
```

```
      (base GeoIP, version figée au chargement) puis rattaché au référentiel Country. L'adresse IP n'est jamais copiée dans
l'OLAP
      (donnée personnelle, RGPD) : seul le pays est conservé.
    - DimGeography = (pays de la transaction, type d'appareil). Ce n'est PAS le pays du client (DimCustomer.country_code) ni
celui
      du marchand (DimMerchant.country_code) : l'écart entre ces pays est un signal d'analyse (transactions transfrontalières,
fraude).
    - Dimension statique (SCD1). Membre par défaut « unknown » (country_code = 'ZZ', device_type = 'unknown') pour les IP non
      résolues, absentes ou hors référentiel : les clés étrangères geography_key des faits sont donc non nulles.

    FUSEAUX HORAIRES :
    - Référence unique : UTC, comme dans l'OLTP (timestamptz). L'ETL ne convertit jamais en heure locale.
    - Horodatages des faits (transaction_timestamp, event_timestamp) : timestamptz, valeurs UTC (jamais de timestamp « nu »
ambigu).
    - date_key = date UTC de l'événement :
        FactTransaction        <- Transaction.created_at
        FactFraudEvent         <- FraudScore.evaluated_at
        FactSubscriptionSnapshot <- date de l'instantané (fin de période de facturation)
        FactAuditEvent         <- DataAccessLog.occurred_at / ChangeAuditLog.occurred_at
        FactDataSubjectRequest <- DataSubjectRequest.received_at / fulfilled_at
        FactSecurityIncident   <- SecurityIncident.detected_at / notified_at / resolved_at
    - Dates de validité SCD2 (valid_from, valid_to), dates d'effet des taux (effective_from, effective_to) et bornes des
      exercices fiscaux : dates UTC. C'est la même date que celle qui sert à choisir le taux de change (règle de
DimReferenceExchangeRate).
    - Requêtes : « aujourd'hui » s'écrit (now() AT TIME ZONE 'UTC')::date, jamais CURRENT_DATE (qui dépend du fuseau de la
session).
    - Analyse par heure : EXTRACT(hour FROM transaction_timestamp AT TIME ZONE 'UTC') sur le fait ; pas de dimension DimTime.
    - Lecture en heure locale du marchand : non retenue. Elle exigerait un fuseau IANA par marchand (Merchant.timezone),
absent de l'OLTP v36,
      et un local_date_key dans les faits ; à n'ajouter que si un besoin métier le justifie.

    INDÉPENDANCE DES DIMENSIONS :
    - Étoile pure : une dimension ne référence jamais une autre dimension ; seules les tables de faits portent des clés
étrangères vers les dimensions.
    - Raison : DimMerchant est en SCD2. Une clé étrangère DimCustomer -> DimMerchant désignerait une version du marchand :
chaque changement
      du marchand obligerait à versionner tous ses clients, produits et plans tarifaires (cascade SCD2).
    - DimCustomer, DimProduct et DimPricingPlan portent merchant_id (clé métier OLTP, uuid, sans lien) : il sert au filtrage
RLS de ces
      dimensions, comme celui de DimMerchant ; le marchand d'une ligne de fait reste merchant_key du fait.
    - DimReferenceExchangeRate identifie la devise source par from_currency_code (même convention que
DimProduct.currency_code).
    - Contrôle ETL : le merchant_id du client, du produit ou du plan d'une ligne de fait doit être celui de son merchant_key.
  '''
}

// ========================================================================
// DIMENSIONS
// ========================================================================

Table DimDate {
  date_key int [pk, note: "Format AAAAMMJJ (ex. 20260723) - clé intelligente, triable et partitionnable"]
  full_date date [not null]
  year smallint [not null]
  quarter smallint [not null]
  month smallint [not null]
  month_name varchar(20) [not null]
  week smallint [not null]
  day_of_month smallint [not null]
  day_of_week smallint [not null]
  day_name varchar(20) [not null]
  is_weekend boolean [not null]

  Note: '''
    Dimension calendrier pré-calculée (jour UTC).

    Évite les fonctions de date coûteuses dans les WHERE (qui bloquent l'élagage de partitions).
    Permet les comparaisons période sur période.

    Abandon des jours fériés : DimDate ne référence aucun pays, donc aucun calendrier de jours fériés unique n'est pertinent.
  '''
}

Table DimMerchant {
  merchant_key int [pk, increment, note: "Clé de substitution SCD2"]
  merchant_id uuid [not null, note: "Identifiant du marchand dans la base transactionnelle. Clé stable du filtrage RLS (une
```

```
merchant_key par version)"]
  legal_name varchar(255) [not null]
  country_code varchar(2) [not null, note: "Merchant.country_code (non nul dans l'OLTP v36)"]
  status varchar(20) [not null]
  valid_from date [not null, note: "Date UTC"]
  valid_to date [note: "NULL = version courante"]
  is_current boolean [not null, default: true]

  Indexes {
    merchant_id
    (merchant_id, is_current)
  }
  Note: '''
    SCD2 (Slowly Changing Dimension Type 2) : un changement de valeur ferme la ligne existante et en crée une nouvelle.
    merchant_id n'est donc plus unique - d'où la nécessité d'une clé de substitution.
    Une transaction reste rattachée à la version du marchand en vigueur à sa date.
  '''
}

Table DimCustomer {
  customer_key int [pk, increment, note: "Clé de substitution SCD2"]
  customer_id uuid [not null, note: "Identifiant du client dans la base transactionnelle"]
  merchant_id uuid [not null, note: "Clé métier du marchand (OLTP), sans clé étrangère : dimensions indépendantes. Filtrage
RLS"]
  segment varchar(50) [note: "nouveau | régulier | VIP | dormant. Calculé à l'ETL, évalué le 1er de chaque mois (D3-C 2.C.3,
ETL-10)"]
  country_code varchar(2) [not null, note: "Customer.country_code (non nul dans l'OLTP v36)"]
  valid_from date [not null, note: "Date UTC"]
  valid_to date
  is_current boolean [not null, default: true]

  Indexes {
    (customer_id, is_current)
    (customer_id)
    (merchant_id)
  }
  Note: "SCD2. Nouvelle version quand le pays ou le segment change (segment : ETL-10, D3-C 2.C.3)."
}

Table DimProduct {
  product_key int [pk, increment, note: "Clé de substitution (SCD1)"]
  product_id uuid [not null]
  merchant_id uuid [not null, note: "Clé métier du marchand (OLTP), sans clé étrangère : dimensions indépendantes. Filtrage
RLS"]
  name varchar(255) [not null]
  unit_price_minor bigint [note: "Prix catalogue en unités mineures de currency_code : unit_price = unit_price_minor /
10^(DimCurrency.decimals)"]
  currency_code varchar(3)

  Indexes {
    (product_id)
    (merchant_id)
  }

  Note: "SCD1 : pas d'historisation (le prix courant suffit, le montant réel est figé dans le fait). Catalogue produits aplati
(pour l'analyse des performances des produits)."
}

Table DimPaymentMethod {
  payment_method_key int [pk, increment, note: "Clé de substitution (SCD1)"]
  type varchar(20) [not null, note: "card | bank_transfer | wallet | direct_debit"]
  network_code varchar(20) [note: "Réseau de la carte (CardNetwork.network_code, OLTP v35) : visa | mastercard | amex |
discover | ... ; null si non applicable"]

  Note: "SCD1. Dimension de faible cardinalité. Aucune donnée de carte sensible."
}

Table DimCurrency {
  currency_key int [pk, increment]
  currency_code varchar(3) [not null, note: "Code ISO 4217 (issu de la base transactionnelle)"]
  name varchar(50) [not null]
  decimals smallint [not null, default: 2, note: "Exposant des unités mineures ISO 4217 (JPY=0, BHD=3, etc.): échelle de tous
les montants *_minor"]

  Indexes {
    currency_code [unique]
```

```
  }

  Note: '''
  Dimension statique. Référentiel des devises. EURO est la devise de référence de l'entrepôt : c'est une constante du modèle.
  '''
}

Table DimPricingPlan {
  pricing_plan_key int [pk, increment, note: "Clé de substitution SCD2"]
  pricing_plan_id uuid [not null, note: "Clé métier issue de l'OLTP"]
  merchant_id uuid [not null, note: "Clé métier du marchand (OLTP), sans clé étrangère : dimensions indépendantes. Filtrage
RLS"]
  commission_rate decimal(5,4) [not null]
  valid_from date [not null, note: "Date UTC"]
  valid_to date [note: "NULL = version courante"]
  is_current boolean [not null, default: true]

  Indexes {
    (pricing_plan_id, is_current)
    (pricing_plan_id)
    (merchant_id)
  }

  Note: '''
    SCD2 - ajoutée en v7, absente du v6.
    Ajoutée pour permettre l'analyse "quel plan tarifaire génère le plus
    de commission" (BR2), et pour rester cohérente avec la traçabilité
    explicite déjà exigée côté OLTP : Transaction.pricing_plan_id est
    conservé pour figer l'historique (traçabilité du plan facturé : règle MCD #22 ;
    cohérence plan/marchand : règle MCD #23 ; fee_amount dérivé : règle MCD #24 ;
    commission_rate indépendant de la devise : règle MCD #25). Les frais fixes par devise (PricingPlanFee)
    ne sont pas repris dans l'entrepôt : la commission réellement facturée est dans FactTransaction.
    Un plan corrigé rétroactivement ne doit pas réécrire l'historique
    des transactions déjà facturées.
  '''
}

Table DimFraudRiskLevel {
  risk_level_key int [pk, increment, note: "Clé de substitution (dimension statique)"]
  risk_level_code varchar(10) [not null, note: "low | medium | high"]
  label varchar(50) [not null]
  requires_manual_review boolean [not null, default: false]
  blocks_transaction boolean [not null, default: false]

  Indexes {
    risk_level_code [unique]
  }
  Note: '''
    Ajoutée en v7 - remplace le varchar risk_level brut du v6 sur
    FactFraudEvent. Porte la même logique de décision que côté OLTP
    (FraudRiskLevel) : requires_manual_review et blocks_transaction
    permettent de filtrer directement sans CASE applicatif.
  '''
}

Table DimSubscriptionStatus {
  subscription_status_key int [pk, increment, note: "Clé de substitution (dimension statique)"]
  status_code varchar(20) [not null, note: "active | past_due | canceled | ..."]
  label varchar(100) [not null]
  is_billable boolean [not null, default: false]
  triggers_dunning boolean [not null, default: false]

  Indexes {
    status_code [unique]
  }
  Note: '''
    Ajoutée en v7, nécessaire au changement de grain de
    FactSubscriptionSnapshot (voir note de cette table).
  '''
}

Table DimRegulation {
  regulation_key int [pk, increment, note: "Clé de substitution (dimension statique)"]
  regulation_code varchar(20) [not null, note: "GDPR | CCPA"]
  label varchar(100) [not null]
  response_deadline_days smallint [not null]
```

```
  Indexes {
    regulation_code [unique]
  }
  Note: '''
    response_deadline_days permet de calculer is_within_deadline par simple comparaison de dates.
  '''
}

Table DimRequestStatus {
  request_status_key int [pk, increment, note: "Clé de substitution (dimension statique)"]
  status_code varchar(20) [not null, note: "received | in_progress | fulfilled | rejected"]
  label varchar(100) [not null]
  is_terminal boolean [not null, default: false]

  Indexes {
    status_code [unique]
  }
  Note: "Ajoutée en v7 - remplace le varchar status brut du v6 sur FactDataSubjectRequest."
}

Table DimIncidentStatus {
  incident_status_key int [pk, increment, note: "Clé de substitution (dimension statique)"]
  status_code varchar(20) [not null, note: "open | investigating | contained | closed"]
  label varchar(100) [not null]
  is_terminal boolean [not null, default: false]

  Indexes {
    status_code [unique]
  }
  Note: "Ajoutée en v7 pour FactSecurityIncident, absente du v6."
}

Table DimReferenceExchangeRate {
  exchange_rate_key int [pk, increment, note: "Clé de substitution (dimension versionnée par exercice)"]
  from_currency_code varchar(3) [not null, note: "Devise locale de la transaction ou de l'abonnement : code ISO 4217, lien
logique avec DimCurrency.currency_code (sans clé étrangère : dimensions indépendantes)"]
  reference_rate decimal(24,12) [not null, note: "Valeur en EUR de 1 unité MAJEURE de la devise source (pas de l'unité
mineure). amount_eur = amount_minor / 10^decimals x reference_rate. Échelle large (24,12) pour les devises de faible valeur
unitaire"]
  rate_basis varchar(50) [not null, note: "Méthode de fixation : moyenne_annuelle | cloture_exercice | taux_budget"]
  fiscal_year smallint [not null, note: "Exercice de rattachement du taux"]
  effective_from date [not null, note: "Date UTC"]
  effective_to date [note: "Date UTC. NULL = taux en vigueur"]
  is_current boolean [not null, default: true]
  source varchar(100) [note: "Origine du taux : BCE, ECB reference rate, taux budgétaire interne"]

  Indexes {
    (from_currency_code, fiscal_year)
    (from_currency_code, is_current)
    (fiscal_year)
  }
  Note: '''
    DIMENSION DE TAUX DE RÉFÉRENCE - un seul taux par paire (devise locale -> EUR) et par exercice.

    Le versionnement par exercice (fiscal_year + effective_from/to) permet de refixer le taux chaque année sans réécrire
l'historique :
    les faits pointent vers la version du taux qui leur est applicable.

    L'analyse de change (impact FX) se traite séparément, en comparant au besoin montant converti au taux de référence et
montant réel.

    Règles d'application :
    - le taux de change retenu est celui de la paire (devise source, exercice) à la date de création de la transaction (UTC) ;
pour un instantané d'abonnement, à la date de l'instantané ;
    - la devise de référence est une constante du modèle (EUR) : elle n'a pas de colonne (pas de to_currency_*) ;
    - la paire EUR -> EUR (taux 1) existe, pour que FactTransaction.exchange_rate_key et
FactSubscriptionSnapshot.exchange_rate_key restent non nuls ;
    - formule de conversion et arrondi : voir « PRÉCISION DES MONTANTS » dans la note du projet.

    Règle de conversion et types (rappel) :
    - montants locaux : bigint en unités mineures (*_minor), échelle DimCurrency.decimals ;
    - reference_rate : decimal(24,12), EUR pour 1 unité MAJEURE ; paire EUR -> EUR = 1 ;
    - amount_eur = amount_minor / 10^decimals x reference_rate, en décimal exact, résultat decimal(18,4) ;
    - arrondi demi-supérieur à la 4e décimale, une seule fois par composante (montant, commission, remboursement) ;
    - exemple (taux fictif) : BHD 12345 (12,345) x 2,4 = 29,6280 EUR ; JPY 5000 (5 000, decimals = 0) x 0,0062 = 31,0000 EUR.
```

```
  '''
}

Table DimGeography {
  geography_key int [pk, increment, note: "Clé de substitution (dimension statique, SCD1)"]
  country_code varchar(2)   [not null, default: 'ZZ', note: "ISO 3166-1 alpha-2 du pays de la transaction. Résolu ETL depuis
Transaction.ip_geolocation. 'ZZ' => 'unknown'"]
  country_name varchar(100) [not null, default: 'unknown', note: "Country.name ; 'unknown' pour le membre par défaut"]
  region varchar(50)        [not null, default: 'unknown', note: "Country.region (nullable côté OLTP) ; 'unknown' si non
renseignée"]
  device_type varchar(20)   [not null, default: 'unknown', note: "mobile | desktop | tablet | unknown (Transaction.device_type
nullable côté OLTP)"]

  Indexes {
    (country_code, device_type) [unique]
  }
  Note: '''
    Dimension dénormalisée (pays + région + appareil dans une seule table) : le schéma en étoile évite les jointures en
cascade.

    Source : Country (référentiel) + Transaction.ip_geolocation + Transaction.device_type.
    Le pays est déduit de ip_geolocation (l'adresse IP de la transaction) à l'ETL ; l'IP n'est pas stockée (donnée
personnelle).

    Statique : une ligne par couple (pays, type d'appareil).
    Les faits référencent le membre par défaut (country_code 'ZZ', country_name 'unknown', region 'unknown', device_type
'unknown')
    au lieu d'une clé nulle (IP absente, privée, non résolue ou pays hors Country).
  '''
}

Table DimTransactionStatus {
  status_key int [pk, increment, note: "Clé de substitution (dimension statique)"]
  status_code varchar(20) [not null, note: "successful | failed | refunded | pending | suspended"]
  is_terminal boolean [not null, default: false]
  allows_refund boolean [not null, default: false]

  Indexes {
    status_code [unique]
  }
  Note: '''
    Renommée depuis DimStatus (v6) en v7, et enrichie de is_terminal/
    allows_refund - déjà présents côté OLTP (TransactionStatus) mais
    absents du v6. Pas de regroupement pré-calculé (v13) :
    status_code et is_terminal suffisent.
  '''
}

Table DimAuditAction {
  audit_action_key int [pk, increment, note: "Clé de substitution (dimension statique)"]
  action_code varchar(30) [not null, note: "read | create | update | delete | export | login | grant | revoke"]
  label varchar(100) [not null]
  action_group varchar(30) [not null, note: "Regroupement pré-calculé : lecture | ecriture | administration. Correspondance
fixe sur action_code (D3-C 2.C.3, ETL-11)"]
  is_sensitive boolean [not null, default: false]
  requires_justification boolean [not null, default: false, note: "AuditAction.requires_justification : motif métier
obligatoire (à comparer à FactAuditEvent.had_justification)"]

  Indexes {
    action_code [unique]
  }
  Note: "Dimension des actions auditables. action_group évite les CASE au reporting."
}

Table DimActor {
  actor_key int [pk, increment, note: "Clé de substitution SCD2"]
  actor_id varchar(255) [not null, note: "Identité applicative de l'acteur"]
  actor_type varchar(20) [not null, note: "user | service | system | merchant_api"]
  is_human boolean [not null, default: false]
  valid_from date [not null, note: "Date UTC"]
  valid_to date
  is_current boolean [not null, default: true]

  Indexes {
    actor_id
    (actor_id, is_current)
```

```
  }
  Note: '''
    SCD2 : conserve le type d'un acteur (actor_type, is_human) à la date de l'accès.
    Exemple : un acteur réaffecté d'un type à un autre reste correctement rattaché sur les accès passés.
  '''
}

// ====================================================================
// TABLES DE FAITS
// Six grains distincts (un par fait) - ne jamais les mélanger dans une seule table.
// ====================================================================

Table FactTransaction {
  transaction_sk bigint [pk, increment, note: "Clé de substitution du fait"]
  transaction_id uuid [not null, note: "Clé métier transactionnelle"]
  date_key int [ref: > DimDate.date_key, not null, note: "Date UTC de transaction_timestamp"]
  merchant_key int [ref: > DimMerchant.merchant_key, not null]
  customer_key int [ref: > DimCustomer.customer_key, not null]
  product_key int [ref: > DimProduct.product_key]
  subscription_id uuid [note: "Clé métier OLTP - null si paiement ponctuel"]
  payment_method_key int [ref: > DimPaymentMethod.payment_method_key, not null]
  currency_key int [ref: > DimCurrency.currency_key, not null]
  exchange_rate_key int [ref: > DimReferenceExchangeRate.exchange_rate_key, not null, note: "Traçabilité : quel taux de
référence a servi à la conversion"]
  geography_key int [ref: > DimGeography.geography_key, not null, note: "Information géographique de la transaction (pays
estimé à partir de ip_geolocation), 'unknown' si non résolu. Ne pas confondre avec DimCustomer.country_code"]
  status_key int [ref: > DimTransactionStatus.status_key, not null, note: "Statut courant de la transaction. Mis à jour après
l'insertion (ETL-02)"]
  pricing_plan_key int [ref: > DimPricingPlan.pricing_plan_key, not null, note: "Traçabilité du plan facturé, cohérent avec
Transaction.pricing_plan_id côté OLTP"]

  amount_minor bigint [not null, note: "Transaction.amount : montant en unités mineures de la devise locale : amount  =
amount_minor / 10^(DimCurrency.decimals)"]
  fee_amount_minor bigint [not null, default: 0, note: "Transaction.fee_amount : commission Stripe en unités mineures de la
devise locale. Reprise telle quelle de l'OLTP, jamais recalculée côté OLAP"]
  refund_amount_minor bigint [not null, default: 0, note: "Refund.amount : remboursement en unités mineures de la devise
locale (0 si aucun) (D3-C 2.C.3, ETL-20). Mis à jour après l'insertion (ETL-02)"]
  amount_eur decimal(18,4) [not null, note: "Montant de la transaction dans la devise de référence : ( amount_minor /
10^(decimals) ) x reference_rate"]
  fee_amount_eur decimal(18,4) [not null, default: 0, note: "Commission prélevée par Stripe en devise de référence : (
fee_amount_minor / 10^(decimals) ) x reference_rate, convertie une seule fois (jamais recalculée)"]
  stripe_revenue_eur decimal(18,4) [not null, note: "Revenu de Stripe en EURO = fee_amount_eur. Colonne DÉRIVÉE : GENERATED
ALWAYS AS (fee_amount_eur) STORED à l'implémentation (DBML ne sait pas l'exprimer), jamais alimentée séparément"]
  merchant_net_revenue_eur decimal(18,4) [not null, note: "Revenu du marchand en EURO = amount_eur - fee_amount_eur -
refund_amount_eur (valeurs déjà converties : identité exacte). Recalculé quand le remboursement change (ETL-02)"]
  refund_amount_eur decimal(18,4) [not null, default: 0, note: "Montant du remboursement en devise de référence, converti une
seule fois (D3-C 2.C.3, ETL-20). Mis à jour après l'insertion (ETL-02)"]
  is_refunded boolean [not null, default: false, note: "Existence d'un Refund pour la transaction, y compris partiel (D3-C
2.C.3, ETL-20). Mis à jour après l'insertion (ETL-02)"]
  is_chargeback boolean [not null, default: false, note: "Existence d'un Chargeback, quel que soit son statut (open, won,
lost) (D3-C 2.C.3, ETL-21). Mis à jour après l'insertion (ETL-02)"]
  is_merchant_initiated boolean [not null, default: false, note: "Prélèvement d'abonnement : sans signal comportemental
client"]
  anomaly_score decimal(5,4) [note: "Score de fraude aplati depuis FraudScore (OLTP) ; NULL si non évalué (D3-C 2.C.3, ETL-
22)"]
  transaction_timestamp timestamptz [not null, note: "Transaction.created_at, en UTC. date_key en est la date UTC"]

  Indexes {
    date_key
    (merchant_key, date_key)
    (customer_key, date_key)
    transaction_id [unique, note: "Dédoublonnage à l'ETL"]
  }
  Note: '''
    Grain : une ligne = une transaction (grain le plus fin).

    Fuseau : transaction_timestamp est en UTC et date_key est la date UTC de cet horodatage (aucune heure locale).

    Conversion : amount_eur est calculé en ETL avec le taux de référence fixe (exchange_rate_key) ;
    Formule et arrondi : note du projet « PRÉCISION DES MONTANTS ».

    Les montants locaux (amount_minor, fee_amount_minor, refund_amount_minor) sont conservés tels quels pour l'audit
    et le rapprochement avec OLTP : SUM(amount_minor) par jour, marchand et devise doit égaler la somme des Transaction.amount
correspondantes.

    Dénormalisation assumée : remboursement, rétrofacturation et score
```

```
    de fraude - tables séparées en OLTP - sont aplatis ici pour éviter
    les jointures.

    Optimisations : partitionnement par date_key (mensuel ou quotidien)
    pour l'élagage de partitions ; stockage colonne ; broadcast join
    des dimensions (petites) sur les nœuds.
  '''
}

Table FactFraudEvent {
  fraud_event_sk bigint [pk, increment, note: "Clé de substitution du fait"]
  transaction_id uuid [not null, note: "Clé métier transactionnelle (cf. FraudScore.transaction_id)"]
  date_key int [ref: > DimDate.date_key, not null]
  merchant_key int [ref: > DimMerchant.merchant_key, not null]
  customer_key int [ref: > DimCustomer.customer_key, not null]
  geography_key int [ref: > DimGeography.geography_key, not null, note: "Hérité de la transaction évaluée (1:1 avec
FraudScore) ; « unknown » si non résolu"]
  risk_level_key int [ref: > DimFraudRiskLevel.risk_level_key, not null]

  anomaly_score decimal(5,4) [not null]
  is_confirmed_fraud boolean [not null, default: false, note: "Vrai si FraudDecision.decision_value = 'fraud' et
decision_status = 'confirmed' (D3-C 2.C.3, ETL-32). Mis à jour après l'insertion (ETL-02)"]
  exposed_amount_eur decimal(18,4) [not null, note: "Montant à risque = FactTransaction.amount_eur de la transaction évaluée,
sans nouvelle conversion (D3-C 2.C.3, ETL-31)"]

  Indexes {
    date_key
    (merchant_key, date_key)
    risk_level_key
    transaction_id [unique, note: "1:1 avec Transaction côté OLTP (FraudScore)"]
  }
  Note: '''
    Grain : un événement d'évaluation de fraude. Alimente les analyses de risque.

    Clés étrangères vers les dimensions :
    - date_key : date UTC de l'évaluation (FraudScore.evaluated_at).
        - C'est un fait transactionnel classique (un seul jalon temporel), et non un instantané cumulatif, comme
FactSecurityIncident ; seule is_confirmed_fraud est mise à jour après l'insertion (D3-C 2.C.3, ETL-02).
        - L'évaluation de fraude est un événement ponctuel, elle ne traverse pas plusieurs étapes dans le temps.
      - merchant_key : obligatoire (not null) : contrairement à SecurityIncident, une évaluation de fraude se rapporte
toujours à une transaction d'un marchand précis, jamais transverse à la plateforme.
      - customer_key : obligatoire (not null) : Transaction.customer_id est non nul dans l'OLTP, une évaluation de fraude se
rapporte donc toujours à un client identifié.
      - geography_key : obligatoire (not null) : le pays (résolu depuis l'IP de la transaction évaluée) et le type d'appareil,
hérités de la transaction ; membre « unknown » si non résolu. Permet de croiser risque et région (inutilisé dans D8.2).
      - risk_level_key : référence DimFraudRiskLevel, qui porte requires_manual_review et blocks_transaction. Même logique que
DimIncidentStatus/DimRequestStatus : la table de référence porte la décision, pas juste une étiquette.

      Mesures
      - anomaly_score : le score brut produit par le modèle de scoring, entre 0 et 1 (decimal(5,4)). Non additif : on ne somme
jamais des scores entre eux, seule la moyenne a un sens (utilisée telle quelle dans aucune requête vue jusqu'ici, mais
servirait par exemple à suivre la dérive moyenne du modèle dans le temps).
      - is_confirmed_fraud : un booléen qui répond à une question différente de risk_level_key : "la fraude s'est-elle
réellement confirmée après coup" (décision de fraude confirmée : FraudDecision avec decision_value = 'fraud' et
decision_status = 'confirmed', cf. D3-C 2.C.3, ETL-32 ; une décision 'defaulted' ne compte pas ; la colonne est mise à jour
quand la décision est confirmée).
      - exposed_amount_eur : le montant en jeu, converti au taux de référence. Additif, contrairement aux deux mesures
précédentes.
  '''
}

Table FactSubscriptionSnapshot {
  snapshot_sk bigint [pk, increment, note: "Clé de substitution du fait"]
  subscription_id uuid [not null, note: "Clé métier transactionnelle"]
  date_key int [ref: > DimDate.date_key, not null, note: "Date UTC de l'instantané, fin de la période de facturation en
cours"]
  merchant_key int [ref: > DimMerchant.merchant_key, not null]
  customer_key int [ref: > DimCustomer.customer_key, not null]
  product_key int [ref: > DimProduct.product_key, not null]
  subscription_status_key int [ref: > DimSubscriptionStatus.subscription_status_key, not null]
  exchange_rate_key int [ref: > DimReferenceExchangeRate.exchange_rate_key, not null, note: "Traçabilité : taux de référence
ayant servi à mrr_eur et stripe_mrr_eur, soit (devise de l'abonnement, exercice de date_key) (ETL-40). La devise de
l'abonnement est celle du taux (from_currency_code) : pas de currency_key dans ce fait"]
  started_date_key int [ref: > DimDate.date_key, not null, note: "Date UTC de Subscription.started_at (début de l'abonnement).
DimDate jouant un second rôle. Source de new_subscriptions"]
  canceled_date_key int [ref: > DimDate.date_key, note: "Date UTC de Subscription.canceled_at : DEMANDE de résiliation du
```

```
client (règle MCD #20), pas la fin de l'abonnement. NULL tant qu'aucune demande n'a été faite, y compris pour une fin par échecs de prélèvement. Valeur connue à la date de l'instantané. Source de churned_count"]

  mrr_eur decimal(18,4) [not null, default: 0, note: "Revenu récurrent mensualisé, converti au taux de référence : montant x facteur (weekly 52/12, monthly 1, yearly 1/12) ; 0 si statut non facturable (D3-C 2.C.3, ETL-40)"]
  stripe_mrr_eur decimal(18,4) [not null, default: 0, note: "Part de commission récurrente revenant à Stripe : commission de la dernière transaction aboutie de l'abonnement, mensualisée comme mrr_eur (D3-C 2.C.3, ETL-41)"]
  failed_attempt_count smallint [not null, default: 0]
  is_active boolean [not null, default: true, note: "Vrai si le statut de l'abonnement est active (D3-C 2.C.3, ETL-42)"]
  days_until_renewal int [note: "Jours entre date_key et current_period_end ; NULL si pas de renouvellement à venir (D3-C 2.C.3, ETL-43)"]

  Indexes {
    (merchant_key, date_key)
    (customer_key, date_key)
    (subscription_id, date_key) [unique, note: "Clé métier du fait : un instantané par abonnement et par date (ETL-03)"]
    (date_key)
    (started_date_key)
    (canceled_date_key)
  }

  Note: '''
    Une ligne = un abonnement, à un instant d'observation (fin de période de facturation) ;
    Conforme à la conception 2.A (Kimball analysis).
    Permet une segmentation des clients par statut d'abonnement (cf. matrice de bus 2.A.6.) et un suivi individuel du risque de désabonnement (failed_attempt_count).

    Les compteurs agrégés sont recalculés à la demande via AggSubscriptionDailyMerchant (réponse rapide aux tableaux de bord ;
    cf. cohérent avec le principe "fait atomique fin + agrégats séparés" déjà appliqué à FactTransaction/ AggRevenueDailyMerchant.

    Snapshots périodiques : mrr_eur est un état des revenus à un instant donné (et non un événement ;
    ne pas sommer cet état sur plusieurs dates pour un même abonnement (semi-additif, additif uniquement sur les dimensions autres que le temps).

    Dates d'un abonnement : date_key = date de l'instantané ; started_date_key = début (Subscription.started_at) ;
    canceled_date_key = demande de résiliation (Subscription.canceled_at). Ces trois colonnes sont trois rôles de DimDate.
    Une résiliation demandée n'arrête pas l'abonnement avant current_period_end (règle MCD #20) : is_active reste vrai jusque-là.

    Le taux de référence fixe permet de fixer l'état MRR (Monthly Recurring Revenue) d'un abonnement indépendamment des fluctuations du taux de change de la monnaie de référence (par rapport à la monnaie de transaction).
  '''
}


Table FactAuditEvent {
  audit_event_sk bigint [pk, increment, note: "Clé de substitution du fait"]
  date_key int [ref: > DimDate.date_key, not null]
  audit_action_key int [ref: > DimAuditAction.audit_action_key, not null]
  actor_key int [ref: > DimActor.actor_key, not null]
  merchant_key int [ref: > DimMerchant.merchant_key]
  customer_key int [ref: > DimCustomer.customer_key]
  geography_key int [ref: > DimGeography.geography_key, not null, note: "Pays d'origine de la requête, résolu à l'ETL depuis DataAccessLog.source_ip ; membre « unknown » pour les lignes issues de ChangeAuditLog (pas d'IP) ou si l'IP n'est pas résolue. device_type est toujours 'unknown' ici"]

  resource_type varchar(50) [not null, note: "Entité consultée ou modifiée"]
  was_authorized boolean [not null, default: true, note: "DataAccessLog.was_authorized ; toujours vrai pour une ligne issue de ChangeAuditLog (D3-C 2.C.3, ETL-60/61)"]
  had_justification boolean [not null, default: false, note: "DataAccessLog : justification non vide. ChangeAuditLog : faux = non journalisée (pas « injustifiée ») ; analyser les justifications sur action_group = lecture (D3-C 2.C.3, ETL-60/61)"]
  event_timestamp timestamptz [not null, note: "DataAccessLog.occurred_at ou ChangeAuditLog.occurred_at, en UTC. date_key en est la date UTC"]

  Indexes {
    (merchant_key, date_key)
    (customer_key, date_key) [note: "Répondre à une demande d'accès RGPD"]
    (actor_key, date_key)
    (was_authorized)
    (date_key)
  }

  Note: '''
    Grain : un événement d'audit (accès ou modification).
    Alimenté depuis DataAccessLog et ChangeAuditLog de l'OLTP.
```

```
    Table en écriture seule. Volume très élevé : partitionnement par
    date_key et archivage à froid au-delà de la durée de conservation.
    Pas de clé métier : un rechargement remplace toutes les lignes d'un même date_key (ETL-03), et non des lignes isolées.

    Géographie (geography_key) :
    - DataAccessLog (accès en lecture) : le pays est résolu à l'ETL depuis source_ip, l'adresse d'origine de la requête
      (même procédé que Transaction.ip_geolocation ; l'adresse IP n'est pas copiée dans l'OLAP).
    - ChangeAuditLog (modifications) : l'OLTP v36 ne porte ni IP ni pays sur ce journal (alimenté par CDC) -> membre « unknown
».
    - device_type : toujours 'unknown' (l'OLTP ne conserve que user_agent, non exploité).
    - Conséquence : une analyse par pays sur cette table ne couvre que les accès, pas les modifications.
      Pour couvrir les modifications, il faudrait porter l'adresse source dans ChangeAuditLog côté OLTP ; non retenu en
l'absence de besoin.
  '''
}

Table FactDataSubjectRequest {
  request_sk bigint [pk, increment, note: "Clé de substitution du fait"]
  request_id uuid [not null, note: "Clé métier OLTP"]
  received_date_key int [ref: > DimDate.date_key, not null]
  fulfilled_date_key int [ref: > DimDate.date_key, note: "Null si non traitée"]
  merchant_key int [ref: > DimMerchant.merchant_key, not null]
  customer_key int [ref: > DimCustomer.customer_key, not null]
  regulation_key int [ref: > DimRegulation.regulation_key, not null]
  request_status_key int [ref: > DimRequestStatus.request_status_key, not null]

  request_type varchar(30) [not null, note: "access | rectification | erasure | portability | objection"]
  processing_days int [note: "Écart entre réception et traitement, en jours"]
  is_within_deadline boolean [note: "Délai légal respecté, dérivable de Regulation.response_deadline_days"]

  Indexes {
    (request_id) [unique, note: "Clé métier : une ligne par demande, mise à jour à chaque jalon (ETL-03)"]
    (received_date_key)
    (merchant_key, received_date_key)
    (regulation_key)
    (is_within_deadline)
  }
  Note: '''
    Grain : une demande d'exercice de droits (instantané cumulatif).
    is_within_deadline est l'indicateur central du rapport RGPD/CCPA.
  '''
}

Table FactSecurityIncident {
  incident_sk bigint [pk, increment, note: "Clé de substitution du fait"]
  incident_id uuid [not null, note: "Clé métier OLTP"]
  detected_date_key int [ref: > DimDate.date_key, not null]
  notified_date_key int [ref: > DimDate.date_key, note: "Null si notification non requise ou non encore faite"]
  resolved_date_key int [ref: > DimDate.date_key, note: "Null si incident non clos"]
  merchant_key int [ref: > DimMerchant.merchant_key, note: "Null si incident transverse à la plateforme"]
  incident_status_key int [ref: > DimIncidentStatus.incident_status_key, not null]
  incident_type varchar(50) [not null, note: "unauthorized_access | data_breach | anomalous_activity | failed_auth_burst"]
  severity varchar(20) [not null, note: "low | medium | high | critical"]
  affected_record_count int
  detection_to_notification_hours decimal(10,2) [note: "Écart entre détection et notification, en heures"]
  is_within_72h_deadline boolean [note: "Point de contrôle direct du délai de notification RGPD (72H)"]

  Indexes {
    (incident_id) [unique, note: "Clé métier : une ligne par incident, mise à jour à chaque jalon (ETL-03)"]
    (detected_date_key)
    (merchant_key, detected_date_key)
    (incident_status_key)
    (is_within_72h_deadline)
  }
  Note: '''
    Complète le volet conformité dans le processus métier P6 (incidents de sécurité),
    aux côtés de FactAuditEvent (accès/modifications) et FactDataSubjectRequest (droits).

    Grain : un incident de sécurité (instantané cumulatif, même famille
    que FactDataSubjectRequest) - une ligne mise à jour au fil des
    jalons detected_at -> notified_at -> resolved_at.

    is_within_72h_deadline matérialise le délai légal RGPD de notification (72h après détection) ;
    même logique que is_within_deadline / FactDataSubjectRequest.
  '''
```

```
}

// ====================================================================
// TABLES D'AGRÉGATS / VUES MATÉRIALISÉES
// Pré-calculées par l'ETL. Répondent instantanément aux requêtes
// de tableau de bord les plus fréquentes.
// ====================================================================

Table AggRevenueDailyMerchant {
  date_key int [ref: > DimDate.date_key]
  merchant_key int [ref: > DimMerchant.merchant_key]
  transaction_count bigint [not null]
  gross_volume_eur decimal(18,4) [not null, note: "Volume encaissé - base du CA marchand"]
  merchant_net_revenue_eur decimal(18,4) [not null, note: "Vue MARCHAND : son chiffre d'affaires net"]
  stripe_revenue_eur decimal(18,4) [not null, note: "Vue STRIPE : commissions perçues"]
  refund_total_eur decimal(18,4) [not null]
  avg_ticket_eur decimal(18,4)

  Indexes {
    (date_key, merchant_key) [pk]
  }
  Note: '''
    Indicateurs de revenus quotidiens des marchands. Couvre l'essentiel des requêtes de dashboard.

    Clé : (date_key, merchant_key).

    Indicateurs :
      - transaction_count : le nombre de transactions du jour concernant le marchand (cf. FactTransaction).
      - gross_volume_eur : le chiffre d'affaire (volume brut encaissé) du marchand et du jour.
      - merchant_net_revenue_eur : le revenu net du marchand et du jour (cf. FactTransaction).
      - stripe_revenue_eur : la somme des commissions prélevées par Stripe sur les transactions concernant ce marchand à ce
jour (cf. FactTransaction).
      - refund_total_eur : la somme des remboursements du jour, l'équivalent agrégé de refund_amount_eur sur le fait atomique.
      - avg_ticket_eur : le panier moyen (gross_volume_eur / transaction_count) pour ce marchand et ce jour.
  '''
}

Table AggRevenueMonthlyMerchantCountry {
  year smallint
  month smallint
  merchant_key int [ref: > DimMerchant.merchant_key]
  country_code varchar(2)
  transaction_count bigint [not null]
  merchant_net_revenue_eur decimal(18,4) [not null]
  stripe_revenue_eur decimal(18,4) [not null]

  Indexes {
    (year, month, merchant_key, country_code) [pk]
  }
  Note: '''
    Indicateurs de revenus mensuels des marchands en fonction des pays.
    Comparabilité inter-pays garantie par le taux de référence fixe.
    Cf. AggRevenueDailyMerchant.
  '''
}

Table AggFraudDailyMerchant {
  date_key int [ref: > DimDate.date_key]
  merchant_key int [ref: > DimMerchant.merchant_key]
  evaluated_count bigint [not null]
  high_risk_count bigint [not null]
  confirmed_fraud_count bigint [not null]
  fraud_rate decimal(5,4)
  exposed_amount_eur decimal(18,4) [not null]

  Indexes {
    (date_key, merchant_key) [pk]
  }
  Note: "Indicateurs de fraude quotidiens par marchand. Alimente l'alerting."
}

Table AggComplianceDailyMerchant {
  date_key int [ref: > DimDate.date_key]
  merchant_key int [ref: > DimMerchant.merchant_key]
  total_access_count bigint [not null]
  sensitive_access_count bigint [not null]
```

```
  unauthorized_attempt_count bigint [not null]
  distinct_actor_count int [not null]
  export_count bigint [not null, note: "Exports de données - point de contrôle sensible"]

  Indexes {
    (date_key, merchant_key) [pk]
  }
  Note: '''
    Indicateurs d'accès quotidiens par marchand.
    Alimente le rapport de conformité automatisé et l'alerting sur les tentatives non autorisées.

    Clé : (date_key, merchant_key)

    Les événements d'audit sans marchand (FactAuditEvent.merchant_key nul) ne sont pas comptés dans cet agrégat par marchand
(D3-C 2.C.3.8, O-08 et O-10).

  Indicateurs :
  - total_access_count : le nombre d'accès aux données de ce marchand à ce jour - tous types d'accès confondus.
  - sensitive_access_count : le nombre d'accès _sensibles_ (directement dérivé de l'attribut transactionnel
AuditAction.is_sensitive).
  - unauthorized_attempt_count : le nombre de tentatives où was_authorized = false côté DataAccessLog : des accès refusés, pas
des accès réussis.
    Alimente directement la règle d'alerte du plan de sécurité (cf. D6: "plus de 3 accès N/A en 10 minutes par le même
acteur").
  - distinct_actor_count : le nombre d'acteurs différents ayant accédé aux données de ce marchand ce jour-là.
    Indicateur de la surface d'exposition du marchand : un marchand consulté par 2 agents de support chaque jour n'a pas le
même profil de risque qu'un marchand consulté par 40.
  - export_count : le nombre d'exportations de données effectués (AuditAction.action_code = 'export').

  Accès Sensible :
  Accès dont le type d'action lui-même a été explicitement marqué comme méritant une vigilance renforcée dans la table de
référence AuditAction (cf. modèle OLTP).
  L'attribut AuditAction.is_sensitive distingue des actions comme export ou delete (potentiellement dommageables, difficiles à
annuler)  d'actions comme une simple routine de lecture.
  Un accès sensible est presque toujours un accès (sur les données transactionnelles) qui a exigé un motif renseigné avant
d'être exécuté.

  Exportations :
  Une exportation de données (typiquement, l'exportation d'un jeu de données entier vers un fichier csv/excel ) est le type
d'accès le plus critique du point de vue de la fuite de données.
  Cette action est notée explicitement dans le modèle transactionnel comme "point de contrôle sensible",
  c'est-à-dire assez importante pour être suivie individuellement, notamment parce qu'un pic soudain d'exportations est le
signal d'alerte le plus révélateur d'une fuite de données en cours.
  '''
}

Table AggSubscriptionDailyMerchant {
  date_key int [ref: > DimDate.date_key]
  merchant_key int [ref: > DimMerchant.merchant_key]
  product_key int [ref: > DimProduct.product_key]
  active_subscriptions int [not null, default: 0, note: "Abonnements actifs au jour date_key, état du dernier instantané
reporté (D3-C 2.C.3, ETL-52)"]
  new_subscriptions int [not null, default: 0, note: "Abonnements dont started_date_key = date_key (ETL-50)"]
  past_due_subscriptions int [not null, default: 0, note: "Prélèvement en échec, relance en cours (triggers_dunning, ETL-52)"]
  churned_count int [not null, default: 0, note: "Demandes de résiliation du jour : canceled_date_key = date_key (ETL-51)"]
  merchant_mrr_eur decimal(18,4) [not null, default: 0]
  stripe_mrr_eur decimal(18,4) [not null, default: 0]
  churn_rate decimal(5,4) [note: "churned_count / active_subscriptions de la veille ; NULL si 0 (ETL-53)"]

  Indexes {
    (date_key, merchant_key, product_key) [pk]
  }
  Note: '''
    NOUVELLE EN V7 - reprend les compteurs agrégés retirés de
    FactSubscriptionSnapshot lors de son changement de grain (v7).
    Calculée par agrégation du fait atomique, pas saisie séparément -
    répond instantanément aux tableaux de bord sans recalcul à la volée.

    Sources des compteurs (date_key de l'agrégat = jour UTC) :
    - new_subscriptions : abonnements distincts dont started_date_key = date_key ;
    - churned_count : abonnements distincts dont canceled_date_key = date_key (demandes de résiliation) ;
    - churn_rate : churned_count / active_subscriptions de la veille (D3-C 2.C.3, ETL-50 à ETL-53).

    Compteurs provisoires : un abonnement n'entre dans le fait qu'à son premier instantané ; les jours concernés sont
recalculés à chaque chargement (D3-C 2.C.3.6).
  '''
```

```
}

Table AggDataSubjectRequestMonthly {
  year smallint
  month smallint
  merchant_key int [ref: > DimMerchant.merchant_key]
  regulation varchar(20) [note: "Dénormalisé pour simplicité de requête - contrairement au fait atomique qui référence
DimRegulation"]
  request_type varchar(30)
  received_count int [not null]
  fulfilled_count int [not null]
  within_deadline_count int [not null]
  avg_processing_days decimal(5,2)

  Indexes {
    (year, month, merchant_key, regulation, request_type) [pk]
  }
  Note: '''
    Synthèse mensuelle des demandes de droits.
    Sortie directe du rapport de conformité.
    Source : FactDataSubjectRequest.
  '''
}

Table AggSecurityIncidentMonthly {
  year smallint
  month smallint
  merchant_key int [ref: > DimMerchant.merchant_key, note: "Null si incident transverse"]
  severity varchar(20)
  incident_count int [not null]
  avg_detection_to_notification_hours decimal(10,2)
  within_72h_count int [not null]

  Indexes {
    (year, month, merchant_key, severity) [unique, note: "merchant_key nullable : pas de clé primaire. À l'implémentation,
NULLS NOT DISTINCT (une seule ligne transverse par mois et sévérité)"]
  }
  Note: '''
    NOUVELLE EN V7, en symétrie avec AggDataSubjectRequestMonthly -
    synthèse mensuelle des incidents de sécurité. Source :
    FactSecurityIncident (nouvelle en v7).

    Clé : (year, month, merchant_key, severity), unique et non primaire, parce que merchant_key est nulle pour un incident
transverse.
    Une clé primaire exige des colonnes non nulles ; en PostgreSQL 15 et plus, l'index unique se déclare NULLS NOT DISTINCT.
  '''
}
```